Partie 1.1
Import

Contexte du projet
Importation des données

L'association NosCités surveille les plateformes web de locations de courte durée pour mesurer l'impact qu'elles ont sur l'offre de logements dans Paris et Lyon.

L'association a choisi d'avoir ses propres serveurs à Paris et à Lyon, et de mettre en place une architecture à partir d'une sauvegarde des données (récupérée suite à un crash).

L'objectif du projet est de:

1. Restaurer la base de données MongoDB
2. Analyser son intégrité par toute une série de requêtes
3. Pérenniser son utilisation pour prévenir de futurs incidents

Chargement par mongoimport de 95885 documents sans erreur:

```
PS C:\Users\ericg\Mon Drive\OpenClassrooms Data Engineer\07_Concevez et analysez une base de données NoSQL\Projet 07>
mongoimport --db locparis --collection listings --type csv --headerline --file listings_Paris.csv
2026-03-13T13:51:30.571+0100    connected to: mongodb://localhost/
2026-03-13T13:51:33.571+0100    [##############..........] locparis.listings    106MB/185MB (57.3%)
2026-03-13T13:51:35.723+0100    [########################] locparis.listings    185MB/185MB (100.0%)
2026-03-13T13:51:35.723+0100    95885 document(s) imported successfully. 0 document(s) failed to import.
```

Partie 1.2

Structure

Structure d'un document et types de données

En vert: texte, booléen ('t'/'f'), date (aaaa-mm-jj)

En jaune: numérique (int, long, float)


```
  _id: ObjectId('69b408521a8c8bf9cb0cfba9'),
  id: 80260,
  listing_url: 'https://www.airbnb.com/rooms/80260',
  scrape_id: Long('20240610195007'),
  last_scraped: '2024-06-13',
  source: 'previous scrape',
  name: "Nice studio in Jourdain's village",
  description: '',
  neighborhood_overview: '',
  picture_url: 'https://a0.muscache.com/pictures/716553/6c14f251_original.jpg',
  host_id: 333548,
  host_url: 'https://www.airbnb.com/users/show/333548',
  host_name: 'Charlotte',
  host_since: '2011-01-03',
  host_location: 'Paris, France',
  host_about: "My name is Charlotte, I'm 26 and I study cinema production. I'll
 be gone the whole year to study in Grenoble and that's my father Jean Luc, who
 will keep an eye on my little studio and take care of you. \n",
  host_response_time: 'N/A',
  host_response_rate: 'N/A',
  host_acceptance_rate: 'N/A',
  host_is_superhost: 'f',
  host_thumbnail_url: 'https://a0.muscache.com/im/users/333548/profile_pic/1350
320876/original.jpg?aki_policy=profile_small',
  host_picture_url: 'https://a0.muscache.com/im/users/333548/profile_pic/135032
0876/original.jpg?aki_policy=profile_x_medium',
  host_neighbourhood: 'Buttes-Chaumont - Belleville',
  host_listings_count: 1,
  host_total_listings_count: 1,
  host_verifications: "['email', 'phone']",
  host_has_profile_pic: 't',
  host_identity_verified: 't',
  neighbourhood: '',
  neighbourhood_cleansed: 'Ménilmontant',
  neighbourhood_group_cleansed: '',
  latitude: 48.87131,
  longitude: 2.38848,
  property_type: 'Entire rental unit',
  room_type: 'Entire home/apt',
  accommodates: 3,
  bathrooms: '',
  bathrooms_text: '1 bath',
  bedrooms: 1,
  beds: '',
  amenities: '["Hangers", "Essentials", "Wifi", "Dishes and silverware", "Dedic
ated workspace", "Hair dryer", "Kitchen", "Shampoo", "Pets allowed", "Heating"]
',
  price: '',
  minimum_nights: 2,
  maximum_nights: 730,
  minimum_minimum_nights: 2,
  maximum_minimum_nights: 2,
  minimum_maximum_nights: 730,
  maximum_maximum_nights: 730,
  minimum_nights_avg_ntm: 2,
  maximum_nights_avg_ntm: 730,
  calendar_updated: '',
  has_availability: 't',
  availability_30: 0,
  availability_60: 0,
  availability_90: 0,
  availability_365: 0,
  calendar_last_scraped: '2024-06-13',
  number_of_reviews: 206,
  number_of_reviews_ltm: 0,
  number_of_reviews_l30d: 0,
  first_review: '2011-04-18',
  last_review: '2021-10-03',
  review_scores_rating: 4.63,
  review_scores_accuracy: 4.61,
  review_scores_cleanliness: 4.75,
  review_scores_checkin: 4.85,
  review_scores_communication: 4.78,
  review_scores_location: 4.61,
  review_scores_value: 4.64,
  license: Long('7512005340473'),
  instant_bookable: 'f',
  calculated_host_listings_count: 1,
  calculated_host_listings_count_entire_homes: 1,
  calculated_host_listings_count_private_rooms: 0,
  calculated_host_listings_count_shared_rooms: 0,
  reviews_per_month: 1.29
```

Partie 1.2
Structure

Avantages du NoSQL pour ce type de BDD

1. Structure: les annonces ont des champs variables et des valeurs parfois absentes.
   Les bases NoSQL acceptent des documents **non strictement structurés**.

2. Scalabilité: les plateformes de location génèrent énormément de données (annonces, hôtes, disponibilités, avis…).
   Les bases NoSQL sont conçues pour **gérer de gros volumes de données**.

3. Flexibilité: contrairement à SQL pas besoin de modifier un schéma pour ajouter de nouveaux champs.
   Les bases NoSQL s'adaptent facilement aux **évolutions de schéma**.

Partie 1.2
Structure

Requêtes préliminaires

Nombre de documents dans la collection:

```
locparis> db.listings.countDocuments()
95885
```

Nombre de logements avec des disponibilités:

```
locparis> db.listings.countDocuments({ availability_365: { $gt: 0 } })
76747
```

availability_365 est utilisé car c'est la variable qui couvre la plus longue période.

Partie 2.1
Mongosh

Nombre d'annonces par type de location

```
locparis> db.listings.aggregate([
|   {
|     $group: {
|       _id: "$room_type",
|       nombre_annonces: { $sum: 1 }
|     }
|   },
|   {
|     $sort: { nombre_annonces: -1 }
|   }
| ])
[
  { _id: 'Entire home/apt', nombre_annonces: 85733 },
  { _id: 'Private room', nombre_annonces: 8975 },
  { _id: 'Hotel room', nombre_annonces: 776 },
  { _id: 'Shared room', nombre_annonces: 401 }
]
```

Agrégation sur la variable room_type avec comptage des occurrences ce chaque catégorie.
On a bien 85733 + 8975 + 776 + 401 = 95885. Pas de valeurs manquantes ou aberrantes.

Partie 2.1
Mongosh

Les 5 annonces avec le plus d'évaluations

```
locparis> db.listings.aggregate([
|   {
|     $project: {
|       _id: 0,
|       name: 1,
|       host_name: 1,
|       number_of_reviews: 1
|     }
|   },
|   {
|     $sort: { number_of_reviews: -1 }
|   },
|   {
|     $limit: 5
|   }
| ])
```

Recherche des 5 valeurs les plus élevées de number_of reviews.
Exclure explicitement _id résultat.

```
[
  {
    name: 'Sweet & cosy room next to Canal Saint Martin 💔',
    host_name: 'Alexandra',
    number_of_reviews: 3067
  },
  {
    name: 'Double/Twin Room, close to Opera and the Louvre with breakfast included',
    host_name: 'David',
    number_of_reviews: 2620
  },
  {
    name: 'Bed in Dorm of 8 Beds "The Big One" in Paris',
    host_name: 'The People',
    number_of_reviews: 2294
  },
  {
    name: 'Comfortable bed in shared rooms of 8 in Paris 12e',
    host_name: 'The People Paris Bercy',
    number_of_reviews: 2105
  },
  {
    name: 'Nice Room for 2 people',
    host_name: 'Hotel De L'Aqueduc',
    number_of_reviews: 2048
  }
]
```

Partie 2.1
Mongosh

Nombre total d'hôtes différents

```
locparis> db.listings.aggregate([
|   {
|     $match: {
|       host_id: { $ne: null }
|     }
|   },
|   {
|     $group: {
|       _id: "$host_id"
|     }
|   },
|   {
|     $count: "nombre_hotes_differents"
|   }
| ])
[ { nombre_hotes_differents: 71979 } ]
```

Comptage des valeurs différentes (et non nulles) de host_id.

Partie 2.1 Nombre de locations réservables instantanément et leur proportion

Mongosh

```
locparis> db.listings.aggregate([
|   {
|     $group: {
|       _id: null,
|       total_annonces: { $sum: 1 },
|       locations_reservables_instantanement: {
|         $sum: { $cond: [{ $eq: ["$instant_bookable", "t"] }, 1, 0]}
|       }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       total_annonces: 1,
|       locations_reservables_instantanement: 1,
|       proportion_pourcent: {
|         $round: [
|           {
|             $multiply: [
|               {
|                 $divide: ["$locations_reservables_instantanement", "$total_annonces"]
|               }, 100]}, 2]
|       }
|     }
|   }
| ])
```

```
[
  {
    total_annonces: 95885,
    locations_reservables_instantanement: 22094,
    proportion_pourcent: 23.04
  }
]
```

Agrégation sur la variable instant_bookable avec comptage des valeurs true.
_id: null pour ne créer qu'un seul groupe (résumé global sur toute la collection).

Partie 2.1
Mongosh

Hôtes ayant plus de 100 annonces et leur proportion

```
locparis> db.listings.aggregate([
|    { $group: { _id: "$host_id", nombre_annonces: { $sum: 1 } } },
|    { $group: { _id: null,
|        total_hotes: { $sum: 1 },
|        nombre_hotes_100_annonces: { $sum: {
|          $cond: [ { $gt: ["$nombre_annonces", 100] }, 1, 0 ] }
|        }
|      }
|    },
|    {
|      $project: {
|        _id: 0,
|        total_hotes: 1,
|        nombre_hotes_100_annonces: 1,
|        proportion_pourcent: { $round: [ { $multiply: [ {
|          $divide: [ "$nombre_hotes_100_annonces", "$total_hotes"] }, 100] }, 5]
|        }
|      }
|    }
| ])
[
  {
    total_hotes: 71979,
    nombre_hotes_100_annonces: 24,
    proportion_pourcent: 0.03334
  }
]
```

Agrégation des host_id avec calcul du nombre d'annonces.
Comptage des host_id et des host_id ayant un nombre d'annonces supérieur à 100.

Partie 2.1
Mongosh

Nombre de super hôtes différents et leur proportion

```
locparis> db.listings.aggregate([
|    { $group: { _id: "$host_id",
|        host_is_superhost: { $first: "$host_is_superhost" } } },
|    { $group: { _id: null,
|        total_hotes: { $sum: 1 },
|        nombre_super_hotes: { $sum: {
|          $cond: [ { $eq: ["$host_is_superhost", "t"] }, 1, 0 ] }
|        }
|      }
|    },
|    { $project: {
|        _id: 0,
|        total_hotes: 1,
|        nombre_super_hotes: 1,
|        proportion_pourcent: { $round: [ { $multiply: [ {
|        $divide: ["$nombre_super_hotes", "$total_hotes"] }, 100] }, 2]
|        }
|      }
|    }
| ])
[
  {
    total_hotes: 71979,
    nombre_super_hotes: 10027,
    proportion_pourcent: 13.93
  }
]
```

Agrégation des host_id avec la variable host_is_superhost. Comptage des host_id et des host_id étant super hôtes.

Partie 2.2
Polars

Taux de réservation moyen par mois par type de logement

```
df_reservation = (df
    .filter(
        (pl.col("availability_365") >= 0) &
        (pl.col("availability_365") <= 365) &
        pl.col("availability_365").is_not_null()
    )
    .group_by("room_type")
    .agg(
        jours_reserves_moyens_par_mois =
        ((365 - pl.col("availability_365")) / 12)
        .mean()
        .round(2)
    )
    .sort("jours_reserves_moyens_par_mois", descending=True)
)
df_reservation
```

room_type	taux_reservation_moyen_mensuel
str	f64
"Private room"	20.93
"Entire home/apt"	19.74
"Shared room"	19.26
"Hotel room"	15.93

La moyenne est prise sur l'année: availability_365 est utilisé pour estimer le nombre de réservations après nettoyage des valeurs nulles et hors plage. Le résultat est groupé et moyenné par room_type.

Partie 2.2
Polars

Médiane du nombre d'avis pour tous les logements

```
df_median = (df
    .filter(
        (pl.col("number_of_reviews") >= 0) &
        pl.col("number_of_reviews").is_not_null()
    )
    .select(
        pl.col("number_of_reviews")
        .median()
        .alias("mediane_nombre_avis")
    )
)
df_median
```

shape: (1, 1)

mediane_nombre_avis
f64
3.0

La médiane est calculée sur la variable number_of_reviews nettoyée des valeurs nulles et négatives.

Partie 2.2
Polars

Médiane du nombre d'avis par catégorie d'hôte

```
df_mediane_superhost = (df
    .filter(
        pl.col("host_is_superhost").is_in(["t", "f"]) &
        (pl.col("number_of_reviews") >= 0) &
        pl.col("number_of_reviews").is_not_null()
    )
    .with_columns(
        pl.when(pl.col("host_is_superhost") == "t")
        .then(pl.lit("Super hôte"))
        .otherwise(pl.lit("Non super hôte"))
        .alias("categorie_hote")
    )
    .group_by("categorie_hote")
    .agg(
        mediane_nombre_avis = pl.col("number_of_reviews")
        .median()
    )
    .sort("mediane_nombre_avis", descending=True)
)
df_mediane_superhost
```

categorie_hote	mediane_nombre_avis
str	f64
"Super hôte"	24.0
"Non super hôte"	2.0

Les médianes sont groupées sur la variable host_is_superhost nettoyée des valeurs non 't' ou 'f' (et reformatée en categorie_hote).
Les médianes sont calculées sur la variable number_of_reviews nettoyée des valeurs nulles et négatives.

Partie 2.2
Polars

Densité de logements par quartier de Paris

```
df_densite_quartier = (df
    .filter(
        pl.col("neighbourhood_cleansed").is_not_null() &
        (pl.col("neighbourhood_cleansed") != "")
    )
    .group_by("neighbourhood_cleansed")
    .agg(nombre_logements = pl.len())
    .with_columns(
        part_logements_pct = (
            pl.col("nombre_logements") * 100
            / pl.col("nombre_logements").sum()
        )
        .round(2)
    )
    .sort("nombre_logements", descending=True)
)
df_densite_quartier
```

neighbourhood_cleansed	nombre_logements	part_logements_pct
str	u32	f64
"Buttes-Montmartre"	10555	11.01
"Popincourt"	8430	8.79
"Vaugirard"	7802	8.14
"Batignolles-Monceau"	6857	7.15
"Entrepôt"	6558	6.84
…	…	…
"Élysée"	2898	3.02
"Hôtel-de-Ville"	2821	2.94
"Palais-Bourbon"	2740	2.86
"Luxembourg"	2701	2.82
"Louvre"	2026	2.11

Les logements sont groupés par la variable neighbourhood_cleansed nettoyée des valeurs nulles et vides. Le nombre de logements dans chaque groupe permet de calculer la densité.

Partie 2.2 Les quartiers avec le plus fort taux de réservation par mois

Polars

```
df_reservation_quartier = (df
    .filter(
        pl.col("neighbourhood_cleansed").is_not_null() &
        (pl.col("neighbourhood_cleansed") != "") &
        pl.col("availability_365").is_not_null() &
        (pl.col("availability_365") >= 0) &
        (pl.col("availability_365") <= 365)
    )
    .group_by("neighbourhood_cleansed")
    .agg(
        taux_reservation_moyen_mensuel = (
            ((365 - pl.col("availability_365")) / 12)
            .mean()
            .round(2)
        ),
        nombre_logements = pl.len()
    )
    .sort("taux_reservation_moyen_mensuel", descending=True)
)
df_reservation_quartier.head(10)
```

neighbourhood_cleansed	taux_reservation_moyen_mensuel	nombre_logements
str	f64	u32
"Ménilmontant"	21.62	5271
"Buttes-Chaumont"	21.21	5465
"Buttes-Montmartre"	21.06	10555
"Entrepôt"	20.99	6558
"Popincourt"	20.95	8430
"Gobelins"	20.76	3304
"Reuilly"	20.61	4003
"Panthéon"	20.18	3010
"Vaugirard"	20.05	7802
"Observatoire"	19.82	3611

Les logements sont groupés par la variable neighbourhood_cleansed nettoyée des valeurs nulles et vides). Le taux de réservations est calculé à partir de la variable availability_365 nettoyée.

Partie 2.3

Power BI

Connecter MongoDB et Power BI

Lancer le BI Connector mongosqld. La collection locparis.listings est accessible à 127.0.0.1:3307

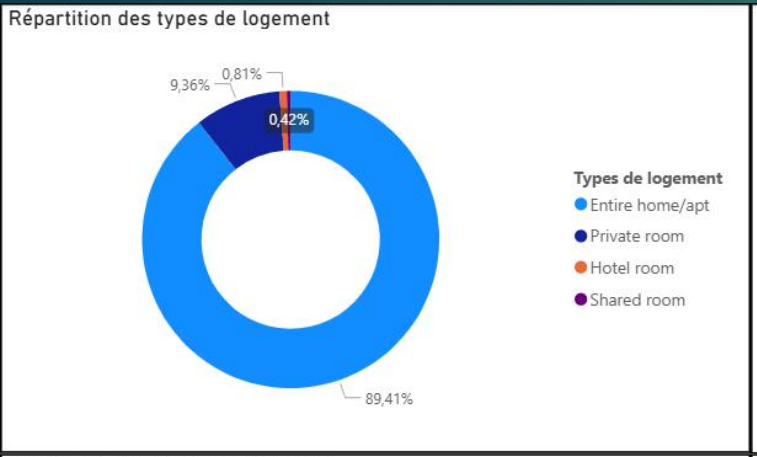
```
mongosqld --mongo-uri "mongodb://localhost:27017" --sampleNamespaces "locparis.listings" --schemaSource "bi_schema"
2026-03-15T17:05:30.146+0100 I CONTROL    [initandlisten] mongosqld starting: version=v2.14.27 pid=34952 host=PC_DUEND
E
2026-03-15T17:05:30.223+0100 I CONTROL    [initandlisten] git version: c34a6bbc85a87049305b3c1f7382fc507c11d6ce
2026-03-15T17:05:30.223+0100 I CONTROL    [initandlisten] OpenSSL version OpenSSL 3.6.0 1 Oct 2025 (built with OpenSSL
 3.6.0 1 Oct 2025)
2026-03-15T17:05:30.223+0100 I CONTROL    [initandlisten] options: {schema: {sample: {namespaces: [locparis.listings]}
, stored: {source: "bi_schema"}}}
2026-03-15T17:05:30.223+0100 I CONTROL    [initandlisten] ** WARNING: Access control is not enabled for mongosqld.
2026-03-15T17:05:30.223+0100 I CONTROL    [initandlisten]
2026-03-15T17:05:30.225+0100 I NETWORK    [initandlisten] waiting for connections at 127.0.0.1:3307
2026-03-15T17:05:30.239+0100 I SCHEMA     [sampler] sampling MongoDB for schema...
2026-03-15T17:05:30.546+0100 I SCHEMA     [sampler] mapped schema for 1 namespace: "locparis" (1): ["listings"]
```

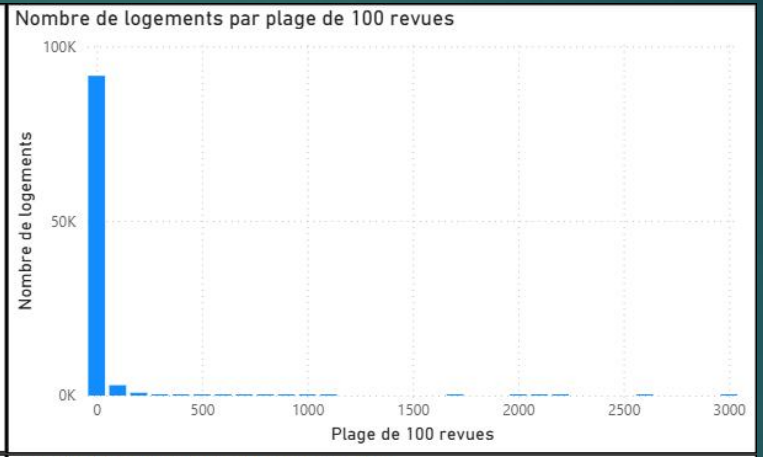
Dans Power Bi connecter la base locparis (par OBDC) et charger la table listings

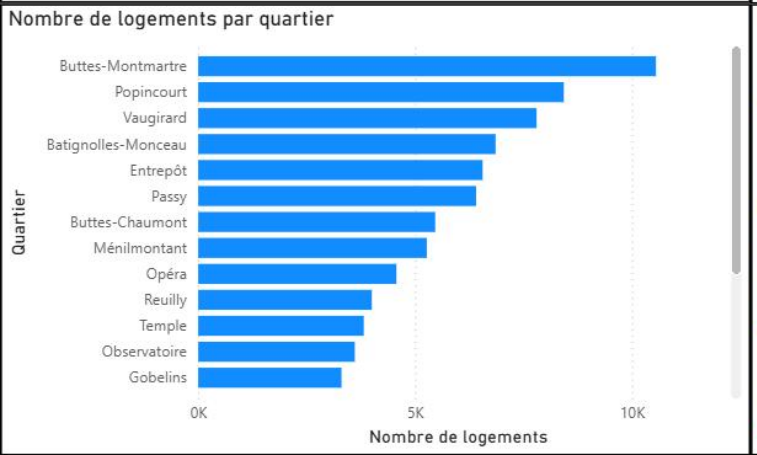
_id	accommodates	
69b408521a8c8bf9cb0d0134	2	["Smoke alarm", "Kitchen", "Wifi"]
69b408521a8c8bf9cb0d05cb	3	["Host greets you", "Extra pillows and blankets", "Dining table", "Refrigerator", "Washer", "Pocket wifi", "Clothing storage: walk-in closet", "Cooking basics", "Wifi", "Dedicated workspa
69b408531a8c8bf9cb0d218a	4	["Smoke alarm", "Cooking basics", "Radiant heating", "Wifi", "Dishwasher", "Dishes and silverware", "Hot water kettle", "Air conditioning", "Coffee maker: espresso machine", "Extra pill
69b408531a8c8bf9cb0d2550	2	["Air conditioning", "Central heating", "Washer", "Cooking basics", "Wifi", "Carbon monoxide alarm", "Dedicated workspace", "Cleaning products", "Free dryer", "Dishes and silverware"
69b408531a8c8bf9cb0d2776	3	["Smoke alarm", "Paid parking on premises", "Wifi", "Essentials", "Drying rack for clothing", "Dedicated workspace", "Kitchen", "TV", "Refrigerator", "Washer", "Bed linens"]
69b408531a8c8bf9cb0d2933	3	["Wifi", "Dedicated workspace", "Kitchen", "TV", "Washer"]
69b408531a8c8bf9cb0d361b	3	["Kitchen", "TV"]
69b408531a8c8bf9cb0d3d7e	2	["Smoke alarm", "Wifi", "Essentials", "Hair dryer", "TV", "Kitchen", "Shampoo", "Pets allowed", "Breakfast", "Washer", "Heating", "Elevator"]

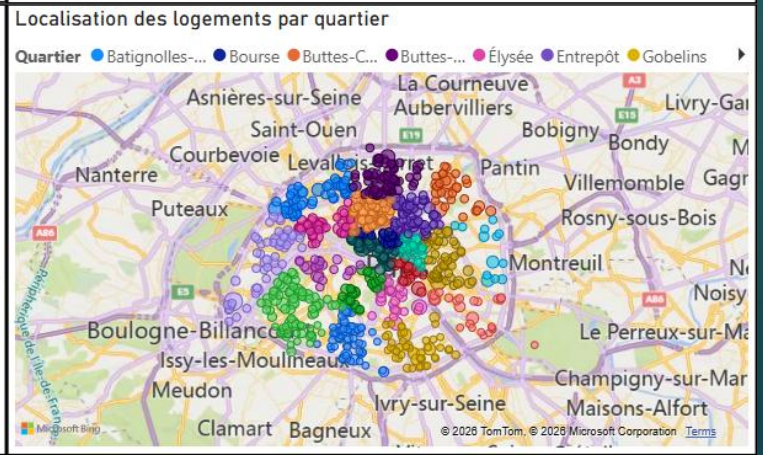
Partie 2.3
Power BI

Exemple de tableau de bord









Graphiques utilisés:

Donut chart
→ Répartition des types de logement

Bar chart
→ Nombre de logements par quartier

Map
→ Localisation des logements par quartier

Histogramme
→ Distribution du nombre d'avis

Partie 3.1
Import Lyon

Ajout de city: "Paris" dans les logements de Paris

```
locparis> use locparis
| db.listings.countDocuments()
already on db locparis
locparis> db.listings.updateMany(
|   { city: { $exists: false } },
|   { $set: { city: "Paris" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 95885,
  modifiedCount: 95885,
  upsertedCount: 0
}
```

```
locparis> db.listings.findOne({ city: "Paris" })
{
  _id: ObjectId('69b408521a8c8bf9cb0cfba9'),
  id: 80260,
            ...
  reviews_per_month: 1.29,
  city: 'Paris'
}
```

La variable city permet de distinguer facilement les logements de Paris de ceux de Lyon pour faire un sharding géographique.

Partie 3.1
Import Lyon

Importer les logements de Lyon avec variable city

Chargement par mongoimport de 9973 documents sans erreur:

```
mongoimport --db locparis --collection listings_lyon_tmp --type csv --headerline --file listings_Lyon.csv
2026-03-18T10:46:58.423+0100    connected to: mongodb://localhost/
2026-03-18T10:46:59.041+0100    9973 document(s) imported successfully. 0 document(s) failed to import.
```

Le chargement dans une collection provisoire listings_lyon_tmp permet de vérifier l'import sans risque.

Ajout de city: "Lyon" dans la collection temporaire et fusion avec la collection principale (mongosh):

```
locparis> db.listings_lyon_tmp.aggregate([
|   { $addFields: { city: "Lyon" } },
|   { $merge: { into: "listings" } }
| ])
```

```
locparis> db.listings.countDocuments({ city: "Lyon" })
9973
locparis> db.listings.countDocuments({ city: "Paris" })
95885
locparis> db.listings.countDocuments()
105858
```

On a bien: 9973 + 95885 = 105858

Suppression de la collection temporaire (mongosh):

```
locparis> db.listings_lyon_tmp.drop()
true
```

Partie 3.1
Import Lyon

Créer un index sur la variable city

La création d'un index sur city permet d'optimiser les requêtes de filtrage et de préparer la mise en place du sharding basé sur cette dimension géographique (clé de distribution):

```
locparis> db.listings.createIndex({ city: 1 })
city_1
```

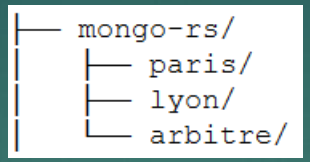
Vérification des indexes:

```
locparis> db.listings.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { city: 1 }, name: 'city_1' }
]
```

Partie 3.2
Réplication

Créer un nœud Paris, un nœud Lyon et un arbitre

Préparer les dossiers de données:



Lancer les 3 instances mongod dans 3 terminaux distincts:

```
mongod --replSet rsLoc --port 27117 --dbpath .\mongo-rs\paris
mongod --replSet rsLoc --port 27118 --dbpath .\mongo-rs\lyon
mongod --replSet rsLoc --port 27119 --dbpath .\mongo-rs\arbitre
```

Se connecter au premier nœud et initialiser le ReplicaSet:

```
test> rs.initiate({
|   _id: "rsLoc",
|   members: [
|     { _id: 0, host: "localhost:27117", priority: 2 },
|     { _id: 1, host: "localhost:27118", priority: 1 },
|     { _id: 2, host: "localhost:27119", arbiterOnly: true }
|   ]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773840766, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773840766, i: 1 })
}
```

Partie 3.2
Réplication

Vérifier la réplication

Créer un document de test dans le nœud Paris:

```
rsLoc [direct: primary] locparis> db.replica_test.insertOne({
|   test: "replication_ok",
|   createdAt: new Date(),
|   source: "primary_27117"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69bab183cb6e23dcd77c2907')
}
```

```
rsLoc [direct: primary] locparis> db.replica_test.find()
[
  {
    _id: ObjectId('69bab183cb6e23dcd77c2907'),
    test: 'replication_ok',
    createdAt: ISODate('2026-03-18T14:06:59.427Z'),
    source: 'primary_27117'
  }
]
```

Vérifier la présence du document de test dans le nœud Lyon:

```
rsLoc [direct: secondary] locparis> db.replica_test.find()
[
  {
    _id: ObjectId('69bab183cb6e23dcd77c2907'),
    test: 'replication_ok',
    createdAt: ISODate('2026-03-18T14:06:59.427Z'),
    source: 'primary_27117'
  }
]
```

Architecture obtenue:

- un **Primaire** à Paris pour centraliser les écritures
- un **Secondaire** à Lyon pour assurer la redondance (read only)
- un **Arbitre** pour garantir un vote majoritaire sans coût de stockage supplémentaire
- cette architecture améliore la **tolérance aux pannes** et assure la **continuité de service**

Partie 3.3
Sharding

Architecture locale

Un ReplicaSet à **3 nœuds pour les config servers**, **1 nœud par shard** et un **routeur**:

- Config servers : cfgRS sur 27217, 27218, 27219 (métadonnées)
- Shard Paris : shardParis sur 27317 (données Paris)
- Shard Lyon : shardLyon sur 27417 (données Lyon)
- Router : mongos sur 27517 (gestionnaire des opérations)

Préparer les dossiers de données:

```
├── mongo-shard/
│   ├── cfg1/
│   ├── cfg2/
│   ├── cfg3/
│   ├── paris/
│   └── lyon/
```

Partie 3.3
Sharding

ReplicaSet des config servers

Lancer les 3 instances cfg mongod dans 3 terminaux distincts:

```
mongod --configsvr --replSet cfgRS --port 27217 --dbpath .\mongo-shard\cfg1
mongod --configsvr --replSet cfgRS --port 27218 --dbpath .\mongo-shard\cfg2
mongod --configsvr --replSet cfgRS --port 27219 --dbpath .\mongo-shard\cfg3
```

Se connecter au premier nœud et initialiser le ReplicaSet:

```
test> rs.initiate({
|   _id: "cfgRS",
|   configsvr: true,
|   members: [
|     { _id: 0, host: "localhost:27217" },
|     { _id: 1, host: "localhost:27218" },
|     { _id: 2, host: "localhost:27219" }
|   ]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773852244, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773852244, i: 1 })
}
```

Partie 3.3
Sharding

Shards Paris et Lyon

Lancer les 2 instances shard mongod dans 2 terminaux distincts:
mongod --shardsvr --replSet shardParis --port 27317 --dbpath .\mongo-shard\paris
mongod --shardsvr --replSet shardLyon --port 27417 --dbpath .\mongo-shard\lyon



Se connecter au shard Paris et l'initialiser:

```
test> rs.initiate({
|   _id: "shardParis",
|   members: [{ _id: 0, host: "localhost:27317" }]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773853491, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773853491, i: 1 })
}
```

Se connecter au shard Lyon et l'initialiser:

```
test> rs.initiate({
|   _id: "shardLyon",
|   members: [{ _id: 0, host: "localhost:27417" }]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773853721, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773853721, i: 1 })
}
```

Partie 3.3
Sharding

Lancer le routeur

Lancer le routeur mongos dans un nouveau terminal:
mongos --configdb cfgRS/localhost:27217,localhost:27218,localhost:27219 --port 27517

Se connecter au terminal mongos et ajouter les shards:

```
[direct: mongos] test> sh.addShard("shardParis/localhost:27317")
{
  shardAdded: 'shardParis',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773855740, i: 57 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773855740, i: 57 })
}
```

```
[direct: mongos] test> sh.addShard("shardLyon/localhost:27417")
{
  shardAdded: 'shardLyon',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773855857, i: 48 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773855857, i: 48 })
}
```

Les shards shardParis et shardLyon sont actifs et le routeur fonctionne

Partie 3.3
Sharding

Activer le sharding pour la base

Dans le routeur mongos lancer le sharding de la base:

```
[direct: mongos] test> sh.enableSharding("locparis")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773906884, i: 22 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773906884, i: 22 })
}
```



Créer la collection shardée sur city:

```
[direct: mongos] test> sh.shardCollection("locparis.listings", { city: 1 })
{
  collectionsharded: 'locparis.listings',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773911011, i: 31 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773911011, i: 30 })
}
```

Partie 3.3
Sharding

Activer le sharding pour la base

Créer les zones:

```
[direct: mongos] test> sh.addShardToZone("shardParis", "ParisZone")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773911297, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773911297, i: 1 })
}
```

```
[direct: mongos] test> sh.addShardToZone("shardLyon", "LyonZone")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773911410, i: 2 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773911410, i: 2 })
}
```

Associer les valeurs de city aux zones:

```
[direct: mongos] test> sh.updateZoneKeyRange(
|   "locparis.listings",
|   { city: "Paris" },
|   { city: "Paris\uFFFF" },
|   "ParisZone"
| )
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773911929, i: 2 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773911929, i: 2 })
}
```

```
[direct: mongos] test> sh.updateZoneKeyRange(
|   "locparis.listings",
|   { city: "Lyon" },
|   { city: "Lyon\uFFFF" },
|   "LyonZone"
| )
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1773912102, i: 2 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1773912102, i: 2 })
}
```

Partie 3.3
Sharding

Charger les données

Chargement par mongoimport des données Paris et Lyon dans des collections provisoires car la variable city n'est pas encore créée:

```
mongoimport --port 27517 --db locparis --collection listings_paris_tmp --type csv --headerline --file listings_Paris.csv
mongoimport --port 27517 --db locparis --collection listings_lyon_tmp --type csv --headerline --file listings_Lyon.csv
```

Ajouter la variable city:

```
[direct: mongos] locparis> db.listings_paris_tmp.updateMany(
|   {},
|   { $set: { city: "Paris" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 95885,
  modifiedCount: 95885,
  upsertedCount: 0
}
```

```
[direct: mongos] locparis> db.listings_lyon_tmp.updateMany(
|   {},
|   { $set: { city: "Lyon" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9973,
  modifiedCount: 9973,
  upsertedCount: 0
}
```

Mettre à jour la base shardée:

```
[direct: mongos] locparis> db.listings_paris_tmp.aggregate([
|   { $merge: { into: "listings" } }
| ])
```

```
[direct: mongos] locparis> db.listings_lyon_tmp.aggregate([
|   { $merge: { into: "listings" } }
| ])
```

Partie 3.3
Sharding

Vérification

Documents chargés:

```
[direct: mongos] locparis> db.listings.countDocuments()
105858
[direct: mongos] locparis> db.listings.countDocuments({ city: "Paris" })
95885
[direct: mongos] locparis> db.listings.countDocuments({ city: "Lyon" })
9973
```

Architecture obtenue:

- **Config servers** : cfgRS
- **Shard Paris** : shardParis
- **Shard Lyon** : shardLyon
- **Routeur** : mongos
- **Collection shardée** : locparis.listings
- **Shard key** : { city: 1 }
- **Zones** : ParisZone, LyonZone

Distribution shard:

```
[direct: mongos] locparis> db.listings.getShardDistribution()
Shard shardParis at shardParis/localhost:27317
{
  data: '327.42MiB',
  docs: 95885,
  chunks: 4,
  'estimated data per chunk': '81.85MiB',
  'estimated docs per chunk': 23971
}
---
Shard shardLyon at shardLyon/localhost:27417
{
  data: '33.23MiB',
  docs: 9973,
  chunks: 1,
  'estimated data per chunk': '33.23MiB',
  'estimated docs per chunk': 9973
}
```

ShardParis contient 95 885docs sur 4 chunks.
ShardLyon contient 9 973 docs sur 1 chunk.
La répartition correspond bien au volume réel de Paris vs Lyon